



Hinweise

Das ist ein bewertetes Übungsblatt, welches für das Bestehen des Modules „Einführung in die Algorithmik“ relevant ist. Bitte beachten Sie folgende Hinweise:

- Erstellen Sie für die Abgabe des Blattes eine lesbare PDF Datei. Erstellen Sie für die Python Programme nur genau eine Datei, in welcher Sie mit Kommentaren kennzeichnen, um welche Aufgabenlösungen es sich handelt. Dies kann beispielsweise wie folgt aussehen:

```
#####
# Übung 1:
#####
# Übung 1-1
def quantumfoo():
```

- Geben Sie dieses Blatt nur in Gruppen von mindestens zwei und maximal vier Personen ab.
- Um Personen für Ihre Übungsgruppe zu finden nutzen Sie bitte die vorgesehenen Foren in StudOn oder ihr Tutorium.
- Benennen Sie die Dateien, welche Sie abgeben wollen nach folgendem Schema, wobei < und > nicht enthalten sein sollen:
 - Schema: hw<ÜbungsNr.>_<Vornamen>_<Nachname>_<Vornamen>_<Nachname>.*
 - Beispiel 3er Gruppe für Übung 2: hw2_Hongjun_Wu_Tanja_Lange_Jonathan_Katz.pdf
- Nutzen Sie zur Abgabe die dafür vorgesehene Umgebung in StudOn.
- Achten Sie auf eine saubere und lesbare Abgabe. Wir empfehlen die Verwendung eines auf T_EX basierenden Textsatzsystems wie beispielsweise L^AT_EX. Den korrigierenden Personen ist es nach eigenem Ermessen gestattet, nicht lesbare Lösungen mit 0 Punkten zu bewerten.
- Wenn Sie in Ihrer PDF-Agabe auf Ihre Programme oder Teile davon verweisen müssen, empfehlen wir die Verwendung des L^AT_EX-Pakets *minted*.
- **Bewertung:** Insgesamt können 50 Punkte pro bewertetem Übungsblatt erzielt werden. Es wird 4 bewertete Blätter geben. Sie brauchen insgesamt mindestens 100 Punkte um die Übung zu bestehen.
- Aus didaktischen Gründen sind die nutzbaren Funktionalitäten von Python eingeschränkt. Die zugelassenen Funktionalitäten finden Sie in den Einführungsfolien zu Python oder als expliziten Hinweis bei den Aufgabenstellungen.

Aufgabe:	1.1	2.1	3.1	3.2	4.1	5.1	5.2	Summe:
Punkte:	7	6	6	5	5	17	4	50
Ergebnis:								



1 Programmfragmente

1.1. Analysieren Sie die asymptotische Laufzeit der folgenden Programmfragmente. Geben Sie dafür jeweils das Laufzeitpolynom an und schätzen Sie dieses in Θ -Notation ab.

(a) (1 Punkt) Einfach:

```
1: for i = 10 to n + 20 by 2 :  
2:     Konstante Operationen
```

(b) (2 Punkte) Unabhängige Verschachtelung:

```
1: for i = 1 to n :  
2:     for j = 5 to 100 by 5 :  
3:         Konstante Operationen
```

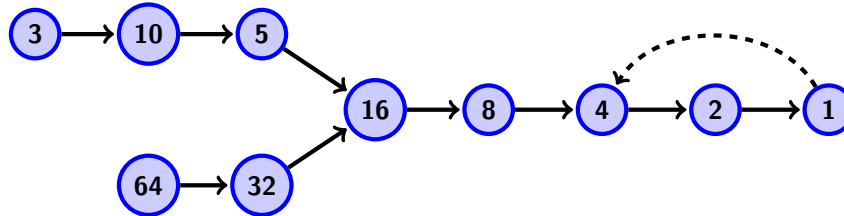
(c) (2 Punkte) Abhängige Verschachtelung:

```
1: for i = 1 to n :  
2:     for j = 1 to i :  
3:         Konstante Operationen
```

(d) (2 Punkte) Knifflig:

```
1: for i = 1 to n by i :  
2:     for j = 1 to i :  
3:         Konstante Operationen
```

2 Ein ungelöstes mathematisches Problem



Das Collatz-Problem, auch bekannt als das $3n + 1$ -Problem, ist eine offene mathematische Frage, die besagt, dass man bei der Anwendung einer bestimmten mathematischen Funktion auf eine beliebige positive ganze Zahl immer auf die Zahl 1 gelangt. Die Funktion besteht aus zwei Fällen: Wenn die Zahl gerade ist, teilt man sie durch 2. Wenn sie ungerade ist, multipliziert man sie mit 3 und addiert 1. Man wiederholt diesen Vorgang mit dem Ergebnis, bis man schließlich die Zahl 1 erreicht.

An dieser Stelle wird nicht erwartet, dass Sie das Problem lösen. Sie sollen sich lediglich mit der algorithmischen Perspektive befassen. Dafür sei die folgende Funktion $cal : \mathbb{N}^+ \rightarrow \mathbb{N}^+$ gegeben:

$$cal(n) = \begin{cases} \frac{n}{2}, & \text{falls } n \equiv 0 \pmod{2} \\ 3 \cdot n + 1, & \text{falls } n \equiv 1 \pmod{2} \end{cases}$$

Weiter sei die Funktion $col : \mathbb{N} \times \mathbb{N}^+ \rightarrow \mathbb{N}^+$ definiert als:

$$col(i, n) = \begin{cases} n, & \text{falls } i = 0 \\ col(col(i - 1, n)) & \text{für } i > 0 \end{cases}$$

- 2.1. (6 Punkte) Analysieren Sie jeweils die durchschnittliche asymptotische Laufzeit der folgenden rekursiven und iterativen Implementierung. Geben Sie dafür jeweils das Laufzeitpolynom an, vereinfachen Sie dieses und schätzen Sie es in Θ -Notation ab.

```

def cal(n):
    if n % 2 == 0:          # Kosten: c1 Anzahl: 1
        return n // 2      # Kosten: c2 Anzahl: 1
    return 3 * n + 1       # Kosten: c3 Anzahl: 1

def col(i, n):
    if i == 0:              # Kosten: c4 Anzahl: 1
        return n           # Kosten: c5 Anzahl: 1
    return cal(col(i-1, n)) # --- Rekursion ---

def col_it(i, n):
    cal_it = lambda n: n//2 if n%2 == 0 else 3*n + 1 # Kosten: c1 Anzahl: 1
    while i > 0:             # Kosten: c2 Anzahl: n+1
        n = cal_it(n)       # Kosten: c3 Anzahl: n
        i -= 1              # Kosten: c4 Anzahl: n
    return n                # Kosten: c5 Anzahl: 1
    
```

3 Divide and ...

Gegeben sei folgendes Programm, welches das Produkt aller Elemente in einem Array berechnet:

`foo(array, start, end)`

Require: Initialer Aufruf mit `start = 0` und `end = length(array) - 1`

```
1: if start == end then
2:   return array[start]
3: mid =  $\lfloor (start + end) / 2 \rfloor$ 
4: return foo(array, start, mid) * foo(array, mid+1, end)
```

- 3.1. (6 Punkte) Analysieren Sie die durchschnittliche asymptotische Laufzeit von `foo`. Geben Sie dafür das Laufzeitpolynom an und nutzen Sie das Mastertheorem, um dieses in Θ -Notation abzuschätzen. Nennen und begründen Sie den Fall des Mastertheorems, welchen Sie nutzen.
- 3.2. (5 Punkte) Zeigen Sie, dass Divide-and-Conquer keinen asymptotischen Laufzeitgewinn bringt. Implementieren Sie dazu eine Methode `def bar(array, start, end)` in Python, welche das Gleiche berechnet wie `foo`, aber keine Rekursion verwendet. Zeigen Sie anschließend, dass die Methoden `def bar(array, start, end)` und `foo` die gleiche durchschnittliche asymptotische Laufzeit haben.

4 Fallunterscheidung und Landau-Notation

Grundsätzlich ist die Fallunterscheidung von Best-, Average- und Worst-Case unabhängig von der Landau Notation. Jeder Fall impliziert ein dazugehöriges Laufzeitpolynom, welches mittels Landau Notation analysiert werden kann.

- 4.1. (5 Punkte) Analysieren Sie die Best- und Worst-Case Laufzeit in Abhängigkeit von n (d.h. ohne Fallunterscheidung für den Münzwurf) des folgenden Algorithmus. Geben Sie diese jeweils in Θ -, Ω - und $\mathcal{O}$ -Notation an. Falls Sie die Laufzeit für eine Notation nicht angeben können, geben Sie dafür eine Begründung an.

`Tricky_Cases(n)`

```
1: if n == 0 then
2:   Mache etwas mit Laufzeit  $\Theta(n^5)$ 
3: else if n mod 2 == 0 then
4:   Wirf eine Münze und speichere Ergebnis in c
5:   if c entspricht Kopf then
6:     Mache etwas mit Laufzeit  $\Theta(n^2)$ 
7:   else
8:     Mache etwas mit Laufzeit  $\Theta(n^4)$ 
9: else
10:  Wirf eine Münze und speichere Ergebnis in c
11:  if c entspricht Kopf then
12:    Mache etwas mit Laufzeit  $\Theta(n^6)$ 
13:  else
14:    Mache etwas mit Laufzeit  $\Theta(n^8)$ 
```

5 Landau-Notation

Die in der Vorlesung vorgestellten Landau-Notationen lassen sich wie folgt definieren:

- $f = \mathcal{O}(g)$ $\exists c > 0 \exists n_0 > 0 \forall n > n_0 : |f(n)| \leq c \cdot |g(n)|$
- $f = \Omega(g)$ $\exists c > 0 \exists n_0 > 0 \forall n > n_0 : c \cdot |g(n)| \leq |f(n)|$
- $f \in \Theta(g)$ $\exists c_1 > 0, c_2 > 0 \exists n_0 > 0 \forall n > n_0 : c_1 \cdot |g(n)| \leq |f(n)| \leq c_2 \cdot |g(n)|$

Alternativ können die Notationen wie folgt anhand der Grenzwertbetrachtung definiert werden:

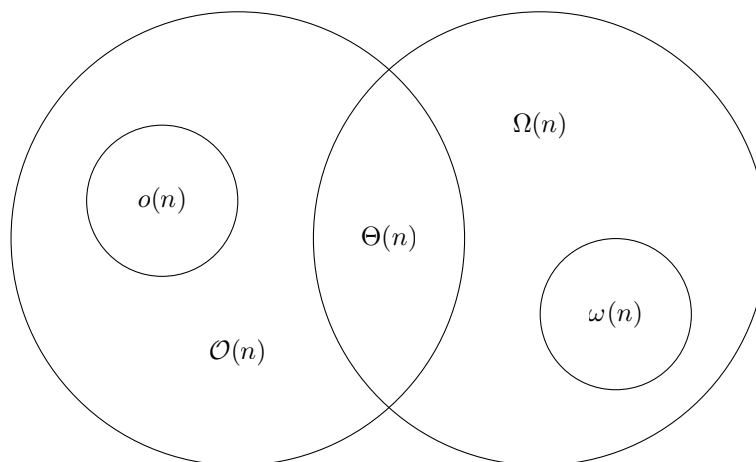
- $f(n) = \mathcal{O}(g(n))$ genau dann wenn $\limsup_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$
- $f(n) = \Omega(g(n))$ genau dann wenn $\liminf_{n \rightarrow \infty} \frac{f(n)}{g(n)} > 0$
- $f(n) = \Theta(g(n))$ genau dann wenn $0 < \liminf_{n \rightarrow \infty} \frac{f(n)}{g(n)} \leq \limsup_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$

Tipp: Sind der Limes Inferior und Limes Superior gleich, so kann die Analyse durch die Limes Betrachtung ersetzt werden. Dies gilt auch immer dann, wenn der Grenzwert einer Funktion existiert. Vereinfacht gesagt kann die Limes-Betrachtung immer dann verwendet werden, wenn ein Polynom vorliegt. Dennoch sollte auch bei der Grenzwertbetrachtung auf mathematische Korrektheit geachtet werden.

5.1. Zeigen oder widerlegen Sie folgende Aussagen:

- (2 Punkte) $30 \cdot n^2 + 104 \cdot n \cdot \log_2 n = \mathcal{O}(n^3)$
- (2 Punkte) $2^n + 40 \cdot n \cdot \log n = \Omega(n)$
- (3 Punkte) Wenn $f(n)$ und $g(n)$ monoton steigende Funktionen sind, dann ist auch $f(g(n))$ monoton steigend.
- (4 Punkte) $f(n) = \mathcal{O}(f(n)^2)$.
- (4 Punkte) Die Θ -Notation erfüllt die Symmetrie Eigenschaft, d.h. wenn $f(n) = \Theta(g(n))$ gilt, dann gilt auch $g(n) = \Theta(f(n))$.
- (2 Punkte) Die $\mathcal{O}$ -Notation erfüllt die Symmetrie Eigenschaft, d.h. wenn $f(n) = \mathcal{O}(g(n))$ gilt, dann gilt auch $g(n) = \mathcal{O}(f(n))$.

5.2. (4 Punkte) Nun gibt es noch weitere Landau Notationen, welche, wie im folgenden Venn-Diagramm beschrieben, in Zusammenhang stehen:





Die kleine o-Notation kann wie folgt definiert werden:

$$f(n) = o(g(n)) \quad \text{genau dann wenn} \quad \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

oder wie folgt:

$$f = o(g) \quad \forall c > 0 \exists n_0 > 0 \forall n > n_0 : |f(n)| < c \cdot |g(n)|$$

Die ω -Notation kann wie folgt definiert werden:

$$f(n) = \omega(g(n)) \quad \text{genau dann wenn} \quad \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

oder wie folgt:

$$f \in \omega(g) \quad \forall c > 0 \exists n_0 > 0 \forall n > n_0 : c \cdot |g(n)| < |f(n)|$$

- (a) Zeigen Sie, dass $f(n) = o(g(n)) \Rightarrow f(n) = \mathcal{O}(g(n))$ gilt.
- (b) Zeigen Sie, dass $f(n) = \omega(g(n)) \Rightarrow f(n) = \Omega(g(n))$ gilt.